

Home Lab Project Week 5, Day 4

Task:

Document the cleartext protocol findings, save the relevant PCAP file and screenshots, and explain the security risks associated with transmitting authentication information without encryption.

Cleartext Protocol Findings — Unencrypted Authentication

Engagement: Home Lab Internal Network Assessment **Lab log:** Week 5, Day 4 **Target:** Ubuntu 192.168.78.5 · **Tester:** Kali 192.168.78.3 **Prepared by:** [Your name], DAE Cybersecurity Program **Date:** [Fill in]

1. Objective

My objective was to document the services on the target that transmit authentication credentials without encryption, capture and preserve the supporting evidence (PCAP files and screenshots), and explain the security risks of sending authentication information in cleartext.

2. Summary of Findings

I found three separate services that transmit credentials in a form readable by anyone able to observe the network traffic.

ID	Service	Port	How credentials are exposed	Captured credential	Severity
CT-01	Telnet	23/tcp	Sent as raw plaintext in the TCP stream	[user:pass]	High
CT-02	FTP (vsftpd)	21/tcp	USER / PASS commands sent as plaintext	[user:pass]	High

CT-03	HTTP Basic Auth	80/tcp	Authorization: Basic header, Base64-encoded only	labuser:labpass	High
-------	-----------------	--------	--	-----------------	------

I identified a common root cause across all three: none of these protocols encrypts the communication channel, so the credentials are protected neither in transit nor by the protocol itself.

3. Detailed Findings

CT-01 — Telnet (port 23)

How it leaks: Telnet transmits the entire session, including the username and password typed at the login prompt, as raw ASCII text — no encoding, no encryption.

How I verified it: I captured the login traffic and reassembled it in Wireshark.

- I ran `sudo tcpdump -i any -A 'tcp port 23' -w telnet-capture.pcap` on the target (alternatively, I could capture in Wireshark on Kali's `eth1` with the filter `tcp.port == 23`).
- I right-clicked a packet and chose **Follow** → **TCP Stream**, and the username and password appeared as plain readable text.

Evidence: `telnet-capture.pcap`, screenshot `telnet-stream.png`.

CT-02 — FTP (port 21)

How it leaks: The FTP protocol sends authentication as two plaintext commands — `USER <name>` and `PASS <password>` — visible directly in the packet payload.

How I verified it:

- I captured on Kali's `eth1` with the filter `host 192.168.78.5 and port 21`.
- I connected with `ftp 192.168.78.5` and logged in using a throwaway account (`labuser`).
- In **Follow** → **TCP Stream**, I could see the `USER` and `PASS` lines in cleartext.

Evidence: `ftp-capture.pcap`, screenshot `ftp-stream.png`.

CT-03 — HTTP Basic Authentication (port 80)

How it leaks: HTTP Basic Auth places credentials in an `Authorization: Basic <string>` header. The string is only **Base64-encoded**, not encrypted — and Base64 is trivially reversible.

How I verified it:

- I captured on Kali's `eth1` with the filter `tcp.port == 80`.
- I sent an authenticated request with `curl -v http://192.168.78.5 -u labuser:labpass`.
- In Wireshark I saw the request header `Authorization: Basic bGFidXNlcjpsYWJwYXNz`, and I decoded it:

bash

```
echo 'bGFidXNlcjpsYWJwYXNz' | base64 -d # → labuser:labpass
```

Evidence: `http-basic-capture.pcap`, screenshot `http-auth-stream.png`.

Key takeaway I want to remember: *Encoding is not encryption*. Base64 changes how data looks, not who can read it — I decoded it in a single command. Only encryption (TLS) actually protects the credential.

4. Saving the Evidence

I saved evidence for each capture so it is preserved and clearly labeled.

Saving the PCAP

I isolated just the relevant traffic before saving:

1. I applied the display filter for the service (e.g. `tcp.port == 21`).
2. I used **File** → **Export Specified Packets**, chose **Displayed**, and saved as `.pcapng` (or `.pcap`).

When I captured with `tcpdump ... -w <name>.pcap`, the file was written directly.

Saving the screenshot

For each finding, I captured the **Follow TCP Stream** window (or the expanded **Authorization** field for HTTP) showing the credentials in cleartext, cropped so the credential is clearly visible.

File naming I used

week5-day4/

- telnet-capture.pcap + telnet-stream.png
- ftp-capture.pcap + ftp-stream.png
- http-basic-capture.pcap + http-auth-stream.png

Consistent naming makes the evidence easy to reference in my final report's evidence index.

5. Security Risks of Transmitting Authentication Without Encryption

This section explains why I consider these findings serious. When credentials cross a network unencrypted, the following risks apply.

1. Credential interception (sniffing). Any party who can observe the network traffic can read the credentials directly. This requires no sophisticated attack — just a packet capture on a shared segment, a compromised host, an untrusted Wi-Fi network, or a device positioned via ARP spoofing. The attacker is passive and leaves no trace.

2. No confidentiality. Encryption's core job is to keep data unreadable to everyone except the intended recipient. Cleartext protocols provide none of this. The password has the same protection sitting on the wire as it would written on a postcard.

3. Encoding provides a false sense of security. HTTP Basic Auth's Base64 encoding *looks* scrambled, which can mislead people into thinking it's protected. It isn't — it's reversible instantly. I think mistaking encoding for encryption is a common and dangerous error.

4. Full account compromise. A captured credential grants the attacker exactly the access that account has. If the account has administrative (`sudo`) rights — as the target's main account does — interception leads directly to complete control of the system.

5. Lateral movement through password reuse. People reuse passwords across systems and services. A single credential captured from an insecure protocol is often the key to many other systems, letting an attacker move sideways through an environment far beyond the original target.

6. Man-in-the-middle and modification. Without encryption there is also no integrity protection, so an attacker positioned in the traffic path can not only read the session but alter it — injecting commands or redirecting the connection.

7. Replay and session hijacking. Captured authentication data or session tokens can sometimes be replayed to impersonate the legitimate user without even needing to crack anything.

8. Compliance and legal exposure. Security standards and regulations (for example PCI-DSS, HIPAA, and GDPR) require that authentication data and sensitive information be encrypted in transit. Using cleartext protocols for authentication is a direct compliance failure with potential legal and financial consequences.

In short: transmitting authentication without encryption means the security of the entire system depends on no one ever watching the network — an assumption that fails the moment an attacker gains any foothold on the same network.

6. Remediation

Each finding has the same fix pattern — replace the plaintext protocol with an encrypted equivalent.

Finding	Insecure protocol	Secure replacement
CT-01	Telnet	SSH (encrypts the whole session)
CT-02	FTP	SFTP or FTPS (FTP over TLS)
CT-03	HTTP Basic Auth	HTTPS (Basic Auth is acceptable <i>only</i> over TLS)

My general recommendation: encrypt authentication and sensitive data in transit everywhere, disable legacy plaintext services, and verify each fix by re-capturing the traffic and confirming it is now unreadable.

7. Evidence Log

- `telnet-capture.pcap + telnet-stream.png` (CT-01)
- `ftp-capture.pcap + ftp-stream.png` (CT-02)
- `http-basic-capture.pcap + http-auth-stream.png` (CT-03)
- `base64 -d` output showing the decoded HTTP credential